TECHNICAL SPECIFICATION

Service Management Platform Software Specification

Functional and technical specification for a category-agnostic service management and fulfillment platform.

Document version	1.0
Status	Reflects the implemented system through Phase 4
Repository	Service Management Platform (monorepo)

Contents

1 Document overview

2 Product overview

3 System architecture

4 Technology stack

5 Domain model

6 Security and access control

7 Functional requirements

8 API specification

9 Data persistence

10 Client applications

11 Configuration

12 Non-functional requirements

13 Testing and quality

14 Delivery phases

15 Out of scope

Appendix A Requirement traceability

Appendix B Source documents

1. Document overview

1.1 Purpose

This document is the consolidated functional and technical specification for the Service Management Platform. It describes the product scope, actors, domain model, authorization model, functional requirements, API surface, data persistence, client applications, configuration, and quality attributes of the system as currently implemented.

It is derived from the implemented source (`backend/`, `admin-web/`, `mobile/`) and the per-phase design notes in `docs/`, and supersedes none of them: the phase notes remain the record of *why* each decision was taken, while this document is the record of *what the system does*.

1.2 Scope

The platform is a multi-sided marketplace for arbitrary, operator-defined services. Customers discover services in a catalog and raise requests. Providers apply to be onboarded, are reviewed by an administrator, and then compete to accept broadcast requests and fulfill them. Administrators manage the catalog, the provider pipeline, users, and role permissions.

In scope for this specification:

- Authentication, token lifecycle, and role/permission authorization
- User and role administration
- Service category and service catalog management
- Provider onboarding, document capture, and review workflow
- Service request broadcast, first-to-accept claiming, and expiry
- Provider job workflow (start, complete, fail) and in-app notifications
- Administrator web portal and customer/provider mobile application

Explicitly out of scope — see Section 15.

1.3 Intended audience

Engineers extending the platform, reviewers assessing the design, QA engineers deriving test coverage, and operators deploying and configuring the system.

1.4 Definitions

Term	Meaning
Customer	An end user who browses the catalog and places service requests. Role `CommonUser`.
Provider	A person or business that applies to fulfill catalog services. Role `ServiceProvider`.
Administrator	An operator of the platform. Role `Administrator`. Seeded, never self-registered.
Service category	A top-level grouping of services, e.g. *Home Cleaning*, *Taxi*.
Service	A bookable unit with a base price and estimated duration, e.g. *Deep Cleaning*, *Sedan*.

Term	Meaning
Service request	A customer's request for one service at one address, broadcast to eligible providers.
Broadcast	The period in which a pending request is visible to all eligible providers.
Eligibility	The composite domain rule determining whether a provider may receive a given request.
Permission	A named capability (e.g. `ManageServices`) granted to roles and carried as a JWT claim.

---

2. Product overview

2.1 Vision

Deliver a low-cost, category-agnostic MVP for service management and fulfillment that an operator can extend to new verticals without code changes.

2.2 Guiding product principles

1. **Category-agnostic by construction.** The domain chain is `ServiceCategory → Service → ServiceRequest → ServiceProvider`. Nothing in the application layer branches on a category or service name. A new vertical is configuration, not code.
2. **Pricing lives on the service, never on the provider.** Vehicle types, cleaning tiers, and similar variants are modelled as ordinary services with their own `BasePrice`. Providers do not quote.
3. **Server-side authorization only.** Clients never decide what a user may do. Every sensitive action is re-validated in the API against role and permission claims.
4. **Deliberate simplicity for the MVP.** One deployable API, one database, no message broker, no cache tier, no realtime transport. Clients poll.

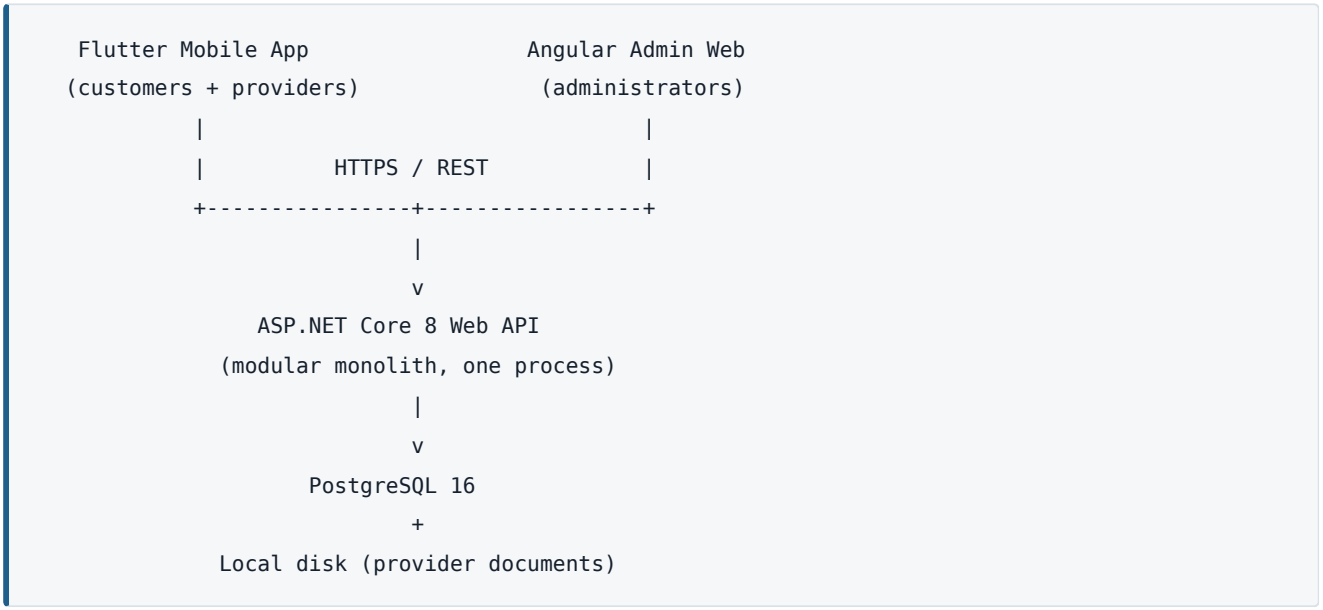
2.3 Actors and their surfaces

Actor	Client	Primary activities
Customer	Flutter mobile app	Browse catalog, place requests, track status, cancel
Provider	Flutter mobile app	Apply and upload documents, set availability, accept jobs, run jobs to completion
Administrator	Angular web portal	Manage catalog and document requirements, review provider applications, manage users and role permissions

---

3. System architecture

3.1 Topology



No client has database access. All business logic and authorization reside in the API.

3.2 Architectural style

A **modular monolith**: one deployable API process over one PostgreSQL database, internally separated into four projects with a strict one-way dependency chain.

Layer	Project	Responsibility	Depends on
Domain	`ServiceManagement.Domain`	Entities, enums, state machine, eligibility rule, permission and role catalogs	nothing
Application	`ServiceManagement.Application`	DTOs, FluentValidation validators, service interfaces, response envelope, exception taxonomy, storage abstraction	Domain
Infrastructure	`ServiceManagement.Infrastructure`	EF Core `DbContext`, migrations, seeding, Identity and JWT wiring, concrete services, file storage, hosted background services	Domain, Application
API	`ServiceManagement.Api`	Thin controllers, middleware, validation filter, Swagger, CORS, rate limiting, DI composition	Application, Infrastructure

Controllers contain no business logic: they resolve the caller's identity, delegate to an application service interface, and wrap the result in the response envelope.

3.3 Deliberate exclusions at MVP scale

Microservices, Kubernetes, Redis, Kafka/RabbitMQ, multiple databases, and SignalR are all excluded. Realtime transport is deferred until polling is demonstrably insufficient.

3.4 Cross-cutting request pipeline

Middleware order in `Program.cs` is significant and specified as follows:

1. Serilog request logging
2. Swagger / Swagger UI (Development environment only)
3. **CORS** — placed before exception handling so that error responses to a browser still carry `Access-Control-Allow-Origin`
4. `ExceptionHandlingMiddleware`
5. Rate limiter (skipped in the `Testing` environment)
6. Authentication
7. Authorization
8. Controller endpoint routing

A `ValidationFilter` registered globally on MVC runs any DI-registered FluentValidation validator against each action argument before the action body executes.

3.5 Response and error contract

Every endpoint returns the envelope `ApiResponse<T>`:

```
{
  "success": true,
  "message": "Request placed. Eligible providers can accept it.",
  "data": { },
  "errors": null
}
```

`ExceptionHandlingMiddleware` maps exceptions to status codes uniformly:

Exception	Status	Body
`AppException` (base)	its `StatusCode`, default `400`	`success:false`, exception message
`NotFoundException`	`404`	message
`ForbiddenException`	`403`	message
`UnauthorizedAppException`	`401`	message
`ConflictException`	`409`	message
`ValidationException` (FluentValidation)	`400`	`"Validation failed."` plus `errors[]` of per-rule messages

Exception	Status	Body
anything else	`500`	generic message; full exception text only in Development/Testing

JSON is camelCase on both directions, case-insensitive on binding, and enums are serialized as camelCase strings while still accepting integers.

---

4. Technology stack

Concern	Choice
API runtime	ASP.NET Core 8 (`net8.0`), nullable reference types and implicit usings enabled
ORM	Entity Framework Core 8 (Npgsql provider)
Database	PostgreSQL 16 (Alpine image for local development)
Identity	ASP.NET Core Identity with `Guid` keys
Tokens	JWT bearer access tokens + persisted rotating refresh tokens
Validation	FluentValidation, invoked by a global MVC action filter
Logging	Serilog (console sink, request logging enrichment)
API documentation	Swashbuckle / Swagger UI with a Bearer security definition
Admin web	Angular 19, Angular Material 19, Reactive Forms, RxJS 7.8, TypeScript 5.7
Admin web tests	Karma + Jasmine
Mobile	Flutter (Dart SDK `>=3.3.0 <4.0.0`), Riverpod, Dio, go_router, `flutter_secure_storage`, `file_picker`
Backend tests	xUnit — unit tests plus `WebApplicationFactory`-based API integration tests
Local infrastructure	Docker Compose (PostgreSQL only)

---

5. Domain model

5.1 Entity catalog

All domain entities except the Identity-derived and pure join types inherit `BaseEntity`, which supplies `Id : Guid` (client-generated), `CreatedAt : DateTime` (UTC), and `UpdatedAt : DateTime?`.

IDENTITY AND ACCESS

Entity	Key fields	Notes
`ApplicationUser`	`FirstName`, `LastName`, `ProfileImageUrl`, `IsActive`	Extends `IdentityUser<Guid>`; mapped to table `Users`
`ApplicationRole`	`Description`	Extends `IdentityRole<Guid>`; mapped to table `Roles`
`Permission`	`Name` (unique), `Description`	The platform capability catalog
`RolePermission`	composite key `(RoleId, PermissionId)`	Many-to-many; cascade delete on both sides
`RefreshToken`	`Token` (unique), `ExpiresAt`, `RevokedAt`	Computed `IsExpired` and `IsActive`; cascade on user delete

CATALOG

Entity	Key fields	Notes
`ServiceCategory`	`Name` (globally unique), `Description`, `ImageUrl`, `IsActive`	Owns services and document requirements
`Service`	`CategoryId`, `Name`, `BasePrice decimal(18,2)`, `IsActive`, `EstimatedDurationMinutes`	Unique index on `(CategoryId, Name)`; category delete is `Restrict`
`CategoryDocumentRequirement`	`CategoryId`, `Name`, `Description`, `IsRequired`, `SortOrder`	Unique index on `(CategoryId, Name)`; cascade from category

PROVIDERS

Entity	Key fields	Notes
`ServiceProviderProfile`	`UserId` (unique), `Description`, `ApprovalStatus` (indexed), `IsActive`, `AvailabilityStatus`, `Rating decimal(3,2)`, `ReviewReason`, `ReviewedAt`, `ReviewedByUserId`	One per provider user; reviewer link is `SetNull` on delete
`ProviderService`	composite key `(ProviderProfileId, ServiceId)`, `Status`, `AppliedAt`, `ReviewedAt`	The provider's *application* to fulfill a service
`ProviderDocument`	`ProviderProfileId`, `DocumentRequirementId`, `OriginalFileName`, `ContentType`, `FileSize`, `StorageKey` (unique)	Unique on `(ProviderProfileId, DocumentRequirementId)` — one file per requirement. Bytes live outside the database

REQUESTS AND FULFILLMENT

Entity	Key fields	Notes
`ServiceRequest`	`CustomerId`, `ServiceId`, `AssignedProviderId?`, `Status`, `RequestedDate`, `ExpiresAt`, `ScheduledDate?`, `AddressId?`, `Notes`, `Price decimal(18,2)`, `CompletedAt?`	Composite index on `(Status, ExpiresAt)` drives the expiry sweep and inbox query; assigned provider is `SetNull` on user delete
`CustomerAddress`	`UserId`, `AddressLine1/2`, `City`, `State`, `PostalCode`, `Latitude?`, `Longitude?`, `IsDefault`	Captured per request in the current flow
`ServiceRequestStatusHistory`	`OldStatus?`, `NewStatus`, `ChangedByUserId`, `Notes`, `ChangedAt`	Append-only audit of lifecycle transitions
`ServiceRequestAttachment`	`FileName`, `FileUrl`, `FileType`, `UploadedByUserId`, `UploadedAt`	Modelled; not exposed by the current API
`UserNotification`	`UserId`, `ServiceRequestId?`, `Title`, `Body`, `IsRead`	Composite index on `(UserId, IsRead)`; request link is `SetNull`
`Review`	`ServiceRequestId` (unique), `CustomerId`, `ProviderId`, `Rating`, `Comment`	Modelled for a later phase; no API surface yet

5.2 Enumerations

Enum	Members	Semantics
`ServiceRequestStatus`	`Pending`, `Accepted`, `Assigned`, `InProgress`, `Completed`, `Cancelled`, `Rejected`, `Failed`, `Expired`	Controlled lifecycle; clients may never set a status directly
`ProviderApprovalStatus`	`Pending`, `Approved`, `Rejected`, `Suspended`	Onboarding review state
`AvailabilityStatus`	`Unavailable`, `Available`, `Busy`	Provider's present willingness to receive work
`ProviderServiceStatus`	`Pending`, `Approved`, `Rejected`	Per-service registration state

5.3 The three orthogonal provider flags

A recurring source of confusion in comparable systems is conflating approval with availability. This platform keeps three independent flags on a provider:

Flag	Owner	Meaning
`ApprovalStatus`	Administrator	Has onboarding been reviewed and accepted?
`IsActive`	Administrator	Operational kill switch, independent of approval. An approved provider can be deactivated without losing approval.
`AvailabilityStatus`	Provider (and the system)	Is the provider willing and free to receive work right now?

5.4 Provider eligibility rule

`ProviderEligibility` is the single authoritative definition of who may be offered a request, expressed both as a predicate and as a composable `IQueryable` filter so the in-memory check and the database query can never drift:

> A provider may receive requests for a service **if and only if** `ApprovalStatus == Approved` **and** `IsActive` **and** `AvailabilityStatus == Available` **and** an approved `ProviderService` row exists for that service.

5.5 Service request state machine

`ServiceRequestStateMachine` is the only gate through which a request's status may change. Permitted transitions:

From	Permitted next states
`Pending`	`Accepted`, `Assigned`, `Cancelled`, `Rejected`, `Expired`
`Accepted`	`Assigned`, `InProgress`, `Cancelled`, `Rejected`
`Assigned`	`InProgress`, `Cancelled`, `Rejected`
`InProgress`	`Completed`, `Failed`, `Cancelled`
`Completed`, `Cancelled`, `Rejected`, `Failed`, `Expired`	*(terminal — no transitions)*

The nominal happy path is therefore `Pending → Accepted → InProgress → Completed`, with `Cancelled`, `Rejected`, `Failed`, and `Expired` as terminal alternatives.

---

6. Security and access control

6.1 Authentication

- ASP.NET Core Identity owns user records, password hashing, and role membership.
- Login issues a **JWT access token** (default lifetime 60 minutes) and a **refresh token** (default lifetime 14 days) persisted in `RefreshTokens`.
- `POST /api/auth/refresh` rotates the refresh token: the presented token is revoked and a new pair is issued. A revoked or expired token is rejected.
- Access-token claims: `NameIdentifier` (user id), `Name` (display name), one `ClaimTypes.Role` claim per role, and one `permission` claim per effective permission.

6.2 Authorization

Authorization is permission-based, not role-based, at the endpoint level. Permissions are resolved from `RolePermission` at token issue and embedded as `permission` claims. Endpoints declare a policy named `Permission:{Name}`, evaluated by a requirement handler against those claims.

Permission catalog (`Permissions.All`):

`ManageUsers`, `ManageServices`, `ManageProviders`, `ManageRequests`, `ManagePricing`, `AssignProviders`, `ViewReports`, `ManageRoles`, `ManagePermissions`, `ManagePlatformConfig`.

Role defaults:

Role	Default permissions
`Administrator`	all ten
`ServiceProvider`	`ManageRequests`
`CommonUser`	none

Where an endpoint is restricted by *actor type* rather than capability — placing a request, or reading a provider inbox — the controller checks role membership and returns `403` with an explanatory message.

6.3 Registration policy

Self-registration is restricted to `CommonUser` and `ServiceProvider`. Administrators are seeded or promoted; the registration endpoint will not mint one. Registering as `ServiceProvider` also creates a `ServiceProviderProfile` in `Pending` approval state.

6.4 Confidentiality of request parties

Identity and contact details are withheld during broadcast. While a request is `Pending`, an eligible provider sees only the service, price, notes, and expiry — not the customer's name, email, phone, or street address. Those fields are populated in `ServiceRequestDto.Customer` / `.Provider` and the full `Address` only after the request has been accepted, and only for the two parties involved.

6.5 Transport, abuse, and secret handling

- CORS is restricted to a configured allowlist (`Cors:AllowedOrigins`), defaulting to the local Angular dev server.
- A fixed-window rate limiter policy (`auth`) permits 20 requests per minute per client IP with no queue, rejecting with `429`.
- Multipart upload size is capped at 11 MB at the framework level, with a 10 MB per-file limit enforced in the storage layer.
- Secrets — the JWT signing key, connection string, and seeded passwords — belong in environment variables or a secret store. The committed `appsettings.json` values are development placeholders and must be replaced before any shared deployment.
- Uploaded documents are served only through an authenticated endpoint that checks ownership or `ManageProviders`; storage keys are not publicly addressable.

---

7. Functional requirements

Requirements are grouped by module and identified as `FR-<module>-<n>`.

7.1 Authentication and accounts (FR-AUTH)

ID	Requirement
FR-AUTH-1	A visitor may register as `CommonUser` or `ServiceProvider` with first name, last name, email, password, optional phone, and role.
FR-AUTH-2	Registration must reject any role outside the self-service set, including `Administrator`.
FR-AUTH-3	Registering as `ServiceProvider` creates a provider profile with `ApprovalStatus = Pending`, `AvailabilityStatus = Unavailable`.
FR-AUTH-4	Login with valid credentials returns an access token, a refresh token, the access token's expiry, and the user's profile including roles and effective permissions.
FR-AUTH-5	Login with invalid credentials returns `401` with a non-enumerating message.
FR-AUTH-6	Presenting a valid refresh token returns a fresh token pair and revokes the presented token.
FR-AUTH-7	A revoked, expired, or unknown refresh token is rejected.
FR-AUTH-8	`GET /api/health` is anonymous and reports service liveness.

Validation. First and last name: required, ≤ 100 characters. Email: required, valid format, ≤ 256 characters. Password: required, 8–128 characters. Phone: ≤ 32 characters when supplied. Role: required and within the permitted set.

7.2 User administration (FR-USER)

ID	Requirement
FR-USER-1	Any authenticated user may read their own profile at `GET /api/users/me`.
FR-USER-2	Listing all users requires `ManageUsers`.
FR-USER-3	Reading or updating a specific user is permitted to that user themselves, or to a caller with `ManageUsers`.

7.3 Roles and permissions (FR-ROLE)

ID	Requirement
FR-ROLE-1	Listing roles requires `ManageRoles`.
FR-ROLE-2	Listing the permission catalog requires `ManagePermissions`.
FR-ROLE-3	Replacing a role's permission set requires `ManagePermissions`.
FR-ROLE-4	Permission changes take effect for a user on their next token issue or refresh.

7.4 Service catalog (FR-CAT)

ID	Requirement
FR-CAT-1	Any authenticated user may read categories and services; customers and providers need catalog reads to transact.
FR-CAT-2	Creating, updating, or deleting a category or service requires `ManageServices`.
FR-CAT-3	`includeInactive` is narrowed server-side to `includeInactive && caller has ManageServices`. A customer passing `?includeInactive=true` still sees only active records and can never enumerate retired services or draft pricing.
FR-CAT-4	A service is visible to a non-administrator only when both the service **and** its category are active.
FR-CAT-5	Deactivating a category deactivates its services in the same transaction, so the two representations cannot drift.
FR-CAT-6	`DELETE /api/service-categories/{id}` always deactivates; categories are never hard-deleted.
FR-CAT-7	`DELETE /api/services/{id}` hard-deletes only when no `ServiceRequest` references the service; otherwise it deactivates, preserving historical requests without orphaned foreign keys.
FR-CAT-8	Duplicate names are rejected twice over: the service layer returns `409` with a readable message, and a unique index backstops it against races. Category names are unique globally; service names are unique within a category.
FR-CAT-9	`GET /api/services` supports filtering by `categoryId`, a case-insensitive `search` across name and description, and `includeInactive`.
FR-CAT-10	A fresh database is seeded with a usable sample catalog; seeding never overwrites operator-managed rows.

Validation. Name: required, ≤ 150 characters. Description: ≤ 2000 characters. `ImageUrl`: ≤ 1024 characters. `BasePrice`: ≥ 0 and < 10,000,000. `EstimatedDurationMinutes`: > 0 when supplied. `SortOrder`: 0–1000.

7.5 Provider onboarding (FR-ONB)

ID	Requirement
FR-ONB-1	A provider may read their own profile, application state, uploaded documents, and outstanding required documents at `GET /api/providers/me`.
FR-ONB-2	A provider may submit or revise their application — a description and the set of catalog services they can fulfill. Each selected service becomes a `ProviderService` row in `Pending`.
FR-ONB-3	Required and optional documents are configured **per category**, so a cook can be asked for a food-safety certificate while a driver is asked for a licence, with no category-specific code.
FR-ONB-4	The onboarding UI loads document requirements from the API, so adding a category never requires a client release.
FR-ONB-5	A provider may upload one file per document requirement. Permitted types are PDF, JPEG, PNG, and WebP, up to 10 MB.
FR-ONB-6	A provider may delete their own document while the application is `Pending` or `Rejected`.

ID	Requirement
FR-ONB-7	Document bytes are stored through `IFileStorage` (local disk at MVP) and never in PostgreSQL; the database holds metadata and an opaque storage key.
FR-ONB-8	A document file is downloadable only by its owning provider or a caller with `ManageProviders`.
FR-ONB-9	A provider may set their own availability only once approved and active.
FR-ONB-10	Unapproved providers retain the normal application tabs; their onboarding state is surfaced on a dedicated **Application** tab rather than locking them out.

Validation. Application description ≤ 2000 characters; the service-id set is validated against the catalog. Rejection and suspension reasons are required, 3–2000 characters.

7.6 Provider review (FR-REV)

ID	Requirement
FR-REV-1	Listing and reading provider profiles requires `ManageProviders`, with filters on approval status and a free-text search.
FR-REV-2	The review list flags providers with missing required documents and shows applied services and document counts.
FR-REV-3	An administrator may approve a provider; approval also approves that provider's pending service applications.
FR-REV-4	An administrator may reject or suspend a provider, and a reason is mandatory in both cases.
FR-REV-5	An administrator may reinstate a previously rejected or suspended provider.
FR-REV-6	An administrator may toggle the operational `IsActive` flag independently of approval.
FR-REV-7	Every review action records the reviewing user and timestamp on the profile.

7.7 Request broadcast and claiming (FR-REQ)

ID	Requirement
FR-REQ-1	Only a `CommonUser` may place a service request; other roles receive `403`.
FR-REQ-2	Placing a request captures the target service, a full address, optional coordinates, and optional notes, and snapshots the service's price onto the request.
FR-REQ-3	A pending request is offered to exactly the providers satisfying the eligibility rule of §5.4.
FR-REQ-4	A request expires if unaccepted within the configured window, default **10 minutes** (`RequestBroadcast:ExpiryMinutes`).
FR-REQ-5	Expiry is applied both lazily on read and by a hosted background sweep running every 30 seconds. The sweep is disabled in the `Testing` environment.
FR-REQ-6	The first valid accept wins. The claim is a **single atomic conditional UPDATE** guarded by `Status = Pending AND AssignedProviderId IS NULL AND ExpiresAt > now`. Two concurrent accepts cannot both succeed; the loser is told the request is no longer available.

ID	Requirement
FR-REQ-7	Accepting sets the winning provider's availability to `Busy`.
FR-REQ-8	A customer may cancel their own request while it is `Pending`, `Accepted`, or `InProgress`, subject to the state machine.
FR-REQ-9	A customer may list their own requests; a provider may list the pending broadcast inbox and, separately, their claimed jobs.
FR-REQ-10	Reading a single request is permitted to its owning customer, its assignee, or a provider currently eligible for it — with field-level redaction per §6.4.

Validation. `ServiceId` required. `AddressLine1` required, ≤ 200 characters; `AddressLine2` ≤ 200. `City` and `State` required, ≤ 100 each. `PostalCode` required, ≤ 20. `Notes` ≤ 4000.

7.8 Provider job workflow (FR-JOB)

ID	Requirement
FR-JOB-1	Only the assigned provider may start, complete, or fail a request.
FR-JOB-2	The claimed-jobs list is distinct from the broadcast inbox: the inbox holds unclaimed pending requests, `GET /api/service-requests/assigned` holds the provider's own jobs.
FR-JOB-3	Completion stamps `CompletedAt`.
FR-JOB-4	On completion, failure, or customer cancellation, the provider's availability is restored to `Available` — but only when they hold no other `Accepted` or `InProgress` job.
FR-JOB-5	Every transition is validated against the state machine and appended to `ServiceRequestStatusHistory` with the acting user.

7.9 Notifications (FR-NOTE)

ID	Requirement
FR-NOTE-1	Status changes write a `UserNotification` row for the affected user. Notifications are database rows, not push messages.
FR-NOTE-2	A user may list their own notifications, mark one read, or mark all read.
FR-NOTE-3	Clients surface notifications by polling and pull-to-refresh; there is no server push channel.

---

8. API specification

Base path `/api`. All endpoints require a bearer token unless marked anonymous. All responses use the `ApiResponse<T>` envelope of §3.5.

8.1 Health and authentication

Method	Path	Authorization
GET	`/api/health`	Anonymous
POST	`/api/auth/register`	Anonymous (`CommonUser` or `ServiceProvider` only)
POST	`/api/auth/login`	Anonymous
POST	`/api/auth/refresh`	Anonymous (valid refresh token)

8.2 Users, roles, permissions

Method	Path	Authorization
GET	`/api/users/me`	Any authenticated user
GET	`/api/users`	`ManageUsers`
GET	`/api/users/{id}`	Self or `ManageUsers`
PUT	`/api/users/{id}`	Self or `ManageUsers`
GET	`/api/roles`	`ManageRoles`
GET	`/api/permissions`	`ManagePermissions`
PUT	`/api/roles/{id}/permissions`	`ManagePermissions`

8.3 Catalog

Method	Path	Authorization
GET	`/api/service-categories`	Any authenticated user
GET	`/api/service-categories/{id}`	Any authenticated user
POST	`/api/service-categories`	`ManageServices`
PUT	`/api/service-categories/{id}`	`ManageServices`
DELETE	`/api/service-categories/{id}`	`ManageServices` — deactivates, never deletes
GET	`/api/services`	Any authenticated user — `categoryId`, `search`, `includeInactive`
GET	`/api/services/{id}`	Any authenticated user
POST	`/api/services`	`ManageServices`
PUT	`/api/services/{id}`	`ManageServices`
DELETE	`/api/services/{id}`	`ManageServices` — deletes if unreferenced, else deactivates

8.4 Document requirements

Method	Path	Authorization
GET	`/api/service-categories/{id}/document-requirements`	Any authenticated user
POST	`/api/service-categories/{id}/document-requirements`	`ManageServices`
PUT	`/api/document-requirements/{id}`	`ManageServices`
DELETE	`/api/document-requirements/{id}`	`ManageServices`

8.5 Provider self-service

Method	Path	Authorization
GET	`/api/providers/me`	Authenticated provider
PUT	`/api/providers/me/application`	Authenticated provider
PUT	`/api/providers/me/availability`	Authenticated provider — approved and active
POST	`/api/providers/me/documents`	Authenticated provider — `multipart/form-data`
DELETE	`/api/providers/me/documents/{documentId}`	Authenticated provider — while `Pending` or `Rejected`
GET	`/api/provider-documents/{id}/file`	Owning provider or `ManageProviders`

8.6 Provider administration

Method	Path	Authorization
GET	`/api/providers`	`ManageProviders` — `approvalStatus`, `search`
GET	`/api/providers/{id}`	`ManageProviders`
POST	`/api/providers/{id}/approve`	`ManageProviders`
POST	`/api/providers/{id}/reject`	`ManageProviders` — reason required
POST	`/api/providers/{id}/suspend`	`ManageProviders` — reason required
POST	`/api/providers/{id}/reinstate`	`ManageProviders`
PUT	`/api/providers/{id}/active`	`ManageProviders`

8.7 Service requests

Method	Path	Authorization
POST	`/api/service-requests`	`CommonUser`
GET	`/api/service-requests/mine`	Authenticated customer
GET	`/api/service-requests/inbox`	`ServiceProvider` — pending broadcast, redacted
GET	`/api/service-requests/assigned`	`ServiceProvider` — claimed jobs

Method	Path	Authorization
GET	`/api/service-requests/{id}`	Owner, assignee, or eligible provider
POST	`/api/service-requests/{id}/accept`	Eligible provider — atomic first-wins claim
POST	`/api/service-requests/{id}/cancel`	Owning customer, while cancellable
POST	`/api/service-requests/{id}/start`	Assigned provider
POST	`/api/service-requests/{id}/complete`	Assigned provider
POST	`/api/service-requests/{id}/fail`	Assigned provider

8.8 Notifications

Method	Path	Authorization
GET	`/api/notifications`	Any authenticated user
POST	`/api/notifications/{id}/read`	Owner
POST	`/api/notifications/read-all`	Any authenticated user

8.9 Principal payloads

```
RegisterRequest  { firstName, lastName, email, password, phoneNumber?, role }
LoginRequest     { email, password }
AuthResponse     { accessToken, refreshToken, accessTokenExpiresAt, user }
UserDto          { id, firstName, lastName, email, phoneNumber?, profileImageUrl?,
                   isActive, roles[], permissions[] }

ServiceDto       { id, categoryId, categoryName, name, description?, basePrice,
                   isActive, estimatedDurationMinutes?, createdAt, updatedAt? }

PlaceServiceRequest { serviceId, addressLine1, addressLine2?, city, state,
                      postalCode, latitude?, longitude?, notes? }
ServiceRequestDto   { id, serviceId, serviceName, categoryName, price, status,
                      requestedDate, expiresAt, completedAt?, notes?,
                      address?, customer?, provider? }   // last three redacted
                                                         // until accepted

ProviderProfileDto  { id, userId, firstName, lastName, email, phoneNumber?,
                      description?, approvalStatus, isActive, availabilityStatus,
                      rating, reviewReason?, reviewedAt?, createdAt, updatedAt?,
                      services[], documents[], missingRequiredDocuments[] }

UserNotificationDto { id, title, body, serviceRequestId?, isRead, createdAt }
```

9. Data persistence

9.1 Mapping conventions

Identity tables are renamed to unprefixed forms — `Users`, `Roles`, `UserRoles`, `UserClaims`, `UserLogins`, `RoleClaims`, `UserTokens`. Monetary values use `decimal(18,2)`; provider rating uses `decimal(3,2)`. Timestamps are UTC throughout.

9.2 Referential integrity

Delete behaviour is chosen per relationship rather than left to convention:

- **Cascade** where the child has no meaning without the parent: role permissions, refresh tokens, provider services, provider documents, category document requirements, status history, notifications.
- **Restrict** where deletion would destroy history: a service referenced by requests, a category owning services, the customer on a request.
- **SetNull** where the reference is advisory: a request's assigned provider, a profile's reviewing administrator, a notification's linked request.

9.3 Indexes of record

Index	Purpose
`Permissions.Name` unique	Capability catalog integrity
`RefreshTokens.Token` unique	Token lookup and replay prevention
`ServiceCategories.Name` unique	Global category uniqueness
`Services (CategoryId, Name)` unique	Per-category service uniqueness
`CategoryDocumentRequirements (CategoryId, Name)` unique	No duplicate requirement per category
`ProviderDocuments (ProviderProfileId, DocumentRequirementId)` unique	One file per requirement
`ProviderDocuments.StorageKey` unique	Storage key collision safety
`ServiceProviderProfiles.UserId` unique	One profile per user
`ServiceProviderProfiles.ApprovalStatus`	Review queue filtering
`ServiceRequests (Status, ExpiresAt)`	Inbox query and expiry sweep
`UserNotifications (UserId, IsRead)`	Unread badge and list
`Reviews.ServiceRequestId` unique	One review per request

9.4 Migration history

Migration	Contents
`InitialCreate`	Identity, permissions, catalog, provider, and request schema
`AddServiceNameUniqueIndex`	Replaces the plain `CategoryId` index with unique `(CategoryId, Name)`
`AddProviderOnboarding`	Approval fields on profiles, status/timestamps on `ProviderServices`, `CategoryDocumentRequirements`, `ProviderDocuments`
`AddRequestBroadcast`	Expiry and assignment columns plus the `(Status, ExpiresAt)` index
`AddProviderWorkflow`	`UserNotifications` and workflow timestamps

Migrations are applied automatically on API startup, and can be applied manually with:

```
cd backend
dotnet ef database update \
  --project src/ServiceManagement.Infrastructure \
  --startup-project src/ServiceManagement.Api
```

9.5 Seed data

`DbSeeder` runs on startup outside the `Testing` environment and is **idempotent and non-destructive** — it never overwrites operator-managed rows.

- **Roles and permissions:** the three roles and the ten permissions, with role defaults.
- **Administrator:** seeded from `Seed:AdminEmail` / `Seed:AdminPassword`.
- **Sample catalog** (only when the catalog is empty): four categories with ten services — *Home Cleaning* (Standard Home Cleaning 49.00, Deep Cleaning 129.00, Kitchen Cleaning 39.00), *Cooking* (Daily Meal Preparation 25.00, Party Catering Assistance 199.00), *Delivery* (Local Package Delivery 12.50, Grocery Pickup 15.00), and *Repairs & Maintenance* (Plumbing Repair 59.00, Electrical Repair 65.00, Appliance Servicing 45.00).
- **Worked example** (added only when missing): category *Taxi* with services *Auto*, *Sedan*, and *SUV*, demonstrating that vehicle types are ordinary priced services rather than a distinct domain concept.
- **Document requirements:** licence, registration, and insurance on *Taxi*; a food-safety certificate on *Cooking* — examples of per-category configuration.
- **Demo accounts:** a customer and a provider, the latter holding a `Pending` application for *Auto* and *Sedan*, ready for an administrator to review.

9.6 File storage

`IFileStorage` abstracts document persistence. The MVP implementation, `LocalFileStorage`, writes beneath `FileStorage:LocalRoot` (default `storage/provider-documents`, resolved against the content root). Because the database stores only an opaque `StorageKey`, substituting object storage requires no schema change and no change to callers.

---

10. Client applications

10.1 Angular administrator portal

A standalone-component Angular 19 application using Angular Material, with lazily-loaded feature routes behind an authentication guard and, where applicable, a permission guard.

Route	Purpose	Guard
`/login`	Credential entry	guest only
`/dashboard`	Landing overview	authenticated
`/users`	User administration	authenticated
`/service-categories`	Category table with create/edit dialogs, show-inactive toggle, per-category service count, and a document-requirements dialog	authenticated
`/services`	Service table with create/edit dialogs, debounced search, category filter, show-inactive toggle	authenticated
`/providers`	Filterable review queue with approval status and missing-document flags	`ManageProviders`
`/providers/:id`	Document review, applied services, approve / reject / suspend / reinstate, `IsActive` toggle	`ManageProviders`
`/my-application`	A provider's own onboarding status	authenticated
`/service-requests`, `/roles`	Reserved placeholders	authenticated

Client-side behaviours required of the portal:

1. `isAuthenticated()` decodes the JWT `exp` claim, so an expired session redirects to login rather than rendering a dashboard that immediately 401s.
2. A response interceptor retries `/api/auth/refresh` once on a `401`, then logs out.
3. Catalog create, edit, and delete controls render only when the user holds `ManageServices` — a usability measure layered over, never in place of, the server-side check.

The development server proxies `/api` to the backend; the production build targets a same-origin `/api` by default, overridable at deploy time.

10.2 Flutter mobile application

One application serving both customers and providers, with Riverpod for state, Dio for transport, go_router for navigation, and `flutter_secure_storage` for tokens. The API base URL is injected at build time via `--dart-define=API_BASE_URL`.

Navigation is a bottom tab shell whose tabs are derived from the signed-in user's role:

Tab	Audience	Content
Home	all	Entry points plus an **Updates** list of notifications

Tab	Audience	Content
Services	all	Category list drilling into that category's services; customers get **Request** on a service card
Requests	all	Customers: their requests with status and cancel. Providers: the broadcast inbox with accept, and **Your jobs** with Start / Complete / Could not complete
Application	providers only	Onboarding status, service selection, and document upload — present even while `Pending`
Profile	all	Account details and sign out

Pending providers are not locked out: they use the same tabs, with onboarding state surfaced on the Application tab. Request and job screens refresh by pull-to-refresh; there is no push channel.

---

11. Configuration

11.1 Settings

Key	Default	Meaning
`ConnectionStrings:DefaultConnection`	local Postgres on port 5433	PostgreSQL connection string
`Jwt:Issuer` / `Jwt:Audience`	`ServiceManagement`	Token issuer and audience
`Jwt:Key`	development placeholder	**Signing key — must be replaced; ≥ 32 characters**
`Jwt:AccessTokenMinutes`	`60`	Access token lifetime
`Jwt:RefreshTokenDays`	`14`	Refresh token lifetime
`Cors:AllowedOrigins`	`http://localhost:4200`, `http://127.0.0.1:4200`	Browser origin allowlist
`RequestBroadcast:ExpiryMinutes`	`10`	Broadcast window before a pending request expires
`FileStorage:LocalRoot`	`storage/provider-documents`	Document directory
`FileStorage:MaxFileBytes`	`10485760` (10 MB)	Per-file upload ceiling
`FileStorage:AllowedContentTypes`	PDF, JPEG, PNG, WebP	Permitted document types
`Seed:AdminEmail` / `Seed:AdminPassword`	`admin@servicemanagement.local` / placeholder	Seeded administrator

Key	Default	Meaning
`Seed:CustomerEmail` / `Seed:ProviderEmail` (+ passwords)	demo accounts	Seeded demo users
`Serilog:MinimumLevel`	`Information`, Microsoft at `Warning`	Log verbosity

All keys are overridable by environment variable using the double-underscore convention, for example `Jwt__Key` — see `.env.example`.

11.2 Environments

Environment	Behaviour
`Development`	Swagger and Swagger UI enabled; full exception detail in error bodies
`Testing`	Expiry sweep, rate limiter, and database seeding all disabled for deterministic tests
Production	Swagger off, generic `500` messages, secrets sourced externally

11.3 Local startup

```
# 1. Database — PostgreSQL on host port 5433 to avoid clashing with a local 5432
docker compose up -d postgres

# 2. API — http://localhost:5080, Swagger at /swagger
cd backend
dotnet restore
dotnet run --project src/ServiceManagement.Api

# 3. Administrator portal — http://localhost:4200
cd admin-web
npm install
npm start

# 4. Mobile client
cd mobile
flutter pub get
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:5080/api
```

---

12. Non-functional requirements

ID	Attribute	Requirement
NFR-1	**Correctness under concurrency**	Concurrent accepts of one request must resolve to exactly one winner. Satisfied by a single conditional `UPDATE`, never a read-then-write.

ID	Attribute	Requirement
NFR-2	**Cost**	The MVP must run as one API process, one PostgreSQL database, and static hosting for the portal — no broker, cache tier, or orchestrator.
NFR-3	**Extensibility**	Adding a service vertical must require no code change: categories, services, prices, and document requirements are all operator-configurable.
NFR-4	**Auditability**	Every request status transition records old status, new status, acting user, and timestamp. Every provider review records reviewer and timestamp.
NFR-5	**Security**	Authorization is enforced server-side on every sensitive action; secrets are externalized; uploads are type- and size-restricted and served only through authenticated endpoints.
NFR-6	**Privacy**	Personal and contact data of request parties is withheld until a request is accepted.
NFR-7	**Observability**	Structured logging via Serilog with request logging; authorization failures are logged at warning level.
NFR-8	**Abuse resistance**	Authentication endpoints are rate-limited per client IP (20/minute, no queue).
NFR-9	**Data integrity**	Business uniqueness rules are enforced by database constraints in addition to application checks.
NFR-10	**Portability of storage**	File bytes are never stored in PostgreSQL; the storage implementation is replaceable without schema or caller changes.
NFR-11	**Interoperability**	The API is self-describing via OpenAPI with a Bearer security scheme.
NFR-12	**Testability**	Domain rules are pure and independently testable; the API is exercised end-to-end through an in-process host with a controllable clock.

---

13. Testing and quality

13.1 Strategy

Two tiers, both xUnit:

Unit tests (`ServiceManagement.UnitTests`) cover the pure domain rules — the highest-value, lowest-cost assertions in the system:

- `ProviderEligibilityTests` — a truth table over the eligibility rule, asserting that each of the four conditions is individually necessary: pending, rejected, and suspended approval all fail; inactive fails; `Unavailable` and `Busy` both fail; and an unregistered service fails.
- `ServiceRequestStateMachineTests` — permitted and forbidden transitions, including that terminal states admit nothing and that `Pending → Completed` is rejected.

Integration tests (`ServiceManagement.IntegrationTests`) drive the real HTTP pipeline through `CustomWebApplicationFactory` in the `Testing` environment, with a `TestClock` substituted for the system clock so expiry is deterministic:

Suite	Coverage
`AuthApiTests`	Registration, login, refresh rotation, role restriction
`CatalogApiTests`	Catalog reads and writes, permission gating, `includeInactive` narrowing, duplicate-name conflicts, soft delete
`ProviderOnboardingApiTests`	Application submission, document upload and access control, approval, rejection, suspension
`RequestBroadcastApiTests`	Placement, inbox visibility and redaction, atomic accept, expiry
`ProviderWorkflowApiTests`	Start, complete, fail, availability restoration, notifications

13.2 Running the suites

```
cd backend && dotnet test     # unit + integration
cd admin-web && npm test      # Karma + Jasmine
```

13.3 Quality invariants to preserve

1. Controllers stay thin — no business logic, no direct status assignment.
2. Status changes go through `ServiceRequestStateMachine`; eligibility goes through `ProviderEligibility`. Neither rule is reimplemented inline.
3. The eligibility predicate and its `IQueryable` form must remain equivalent.
4. New uniqueness rules get both an application check and a database constraint.
5. New validators are registered in DI; the global filter then enforces them.

---

14. Delivery phases

Phase	Scope	Status
1 — Foundation	Solution structure, domain model, JWT auth with Identity, RBAC, Angular admin shell, Flutter shell, auth/RBAC tests	Complete
2 — Service catalog	Admin CRUD for categories and services, base pricing and duration, catalog browsing for customers and providers, visibility narrowing, soft delete, sample seed catalog, validation pipeline activation	Complete
3A — Provider onboarding	Applications, per-category document requirements, uploads via `IFileStorage`, admin review UI, approval independent of `IsActive` and availability, eligibility rule	Complete
3B — Request broadcast	Placement with address, eligible-provider inbox, atomic first-to-accept, contact redaction, 10-minute expiry with background sweep	Complete
4 — Provider workflow	Assigned-jobs list, start / complete / fail, availability restoration, in-app notifications	Complete

Phase	Scope	Status
5 — Tracking and reviews	Customer tracking, request history, reviews and ratings	Planned

14.1 Phase 5 and beyond

The schema already anticipates the next phase: `Review` is modelled with a unique constraint of one review per request, `ServiceRequestAttachment` is modelled, and `CustomerAddress` carries latitude and longitude for future tracking. None of these have an API surface yet.

---

15. Out of scope

The following are deliberately excluded from the current system:

- **Payments, invoicing, and payouts** — no money moves through the platform.
- **Live GPS tracking and maps** — coordinates are captured but not tracked.
- **Reviews and ratings** — modelled, not exposed (Phase 5).
- **Email, SMS, and push notifications** — notifications are in-app rows only.
- **Realtime transport (SignalR/WebSockets)** — clients poll; to be re-evaluated only if polling proves insufficient.
- **Per-provider pricing or quoting** — price is a property of the service.
- **Scheduled or recurring bookings** — `ScheduledDate` is modelled but unused; the current flow is immediate broadcast.
- **Administrator-initiated assignment** — the `AssignProviders` permission and the `Assigned` status exist for a future dispatch flow; today providers self-claim.
- **Multi-tenancy, i18n/l10n, and object storage** — single tenant, English, local disk.

Appendix A — Requirement traceability

Module	Requirements	Primary implementation	Primary tests
Authentication	FR-AUTH-1…8	`AuthController`, `AuthService`, `JwtTokenService`	`AuthApiTests`
Users	FR-USER-1…3	`UsersController`, `UserService`	`AuthApiTests`
Roles	FR-ROLE-1…4	`RolesController`, `RoleService`	`AuthApiTests`
Catalog	FR-CAT-1…10	`ServiceCategoriesController`, `ServicesController`, `ServiceCategoryService`, `ServiceCatalogService`	`CatalogApiTests`

Module	Requirements	Primary implementation	Primary tests
Onboarding	FR-ONB-1…10	`ProvidersController`, `ProviderDocumentsController`, `ProviderOnboardingService`, `LocalFileStorage`	`ProviderOnboardingApiTests`
Review	FR-REV-1…7	`ProvidersController`, `ProviderOnboardingService`	`ProviderOnboardingApiTests`
Requests	FR-REQ-1…10	`ServiceRequestsController`, `ServiceRequestService`, `RequestExpiryHostedService`, `ProviderEligibility`	`RequestBroadcastApiTests`, `ProviderEligibilityTests`
Job workflow	FR-JOB-1…5	`ServiceRequestService`, `ServiceRequestStateMachine`	`ProviderWorkflowApiTests`, `ServiceRequestStateMachineTests`
Notifications	FR-NOTE-1…3	`NotificationsController`, `NotificationService`	`ProviderWorkflowApiTests`

Appendix B — Source documents

Document	Contents
`README.md`	Orientation, stack summary, quick start
`docs/ARCHITECTURE.md`	Architectural style, layering, authorization approach
`docs/PHASE1.md`	Foundation goals, assumptions, backend layout
`docs/PHASE2.md`	Catalog design decisions and visibility rules
`docs/PHASE3A.md`	Onboarding model, the three provider flags, eligibility
`docs/PHASE3B.md`	Broadcast, atomic accept, redaction, expiry
`docs/PHASE4.md`	Job workflow, availability restoration, notifications